VIGOR TRACK - Complete Setup & Execution Guide

Table of Contents

- Disease Prediction Mobile Application
- Table of Contents
- Project Overview
- Architecture
- Prerequisites
- Backend Setup
- Frontend Setup
- Running the Application
- Testing & Usage
- Troubleshooting
- Create virtual environment
- Activate it
- Windows:
- Mac/Linux:
- Install packages
- Delete cache and dependencies
- Reinstall
- Clear Metro bundler cache
- Install ngrok
- Expose Flask
- Use ngrok URL in App.tsx
- Example: http://abc123.ngrok.io/predict
  - Development Tips
  - Next Steps
  - Appendix
- Get IP address
- Check port usage
- Test connectivity

Disease Prediction Mobile Application

Table of Contents

1. Project Overview
2. Architecture
3. Prerequisites
4. Backend Setup
5. Frontend Setup
6. Running the Application
7. Testing & Usage
8. Troubleshooting
9. Development Tips

Project Overview

VIGOR TRACK is a machine learning-powered mobile application that predicts diseases based on user-selected symptoms. The application consists of:

- **Backend**: Python Flask API with Random Forest ML model
- **Frontend**: React Native mobile app built with Expo
- **Database**: CSV files containing disease information
- **Model**: Trained on 4,920 symptom-disease records covering 41 diseases

Key Features

- 133 symptoms available for selection
- Real-time disease prediction
- Disease descriptions and precautions
- User-friendly mobile interface
- Cross-platform support (Android & iOS)

Architecture

System Components

1. **Mobile App (React Native)**
   - User interface for symptom selection
   - Axios HTTP client for API communication
   - Modal popup for displaying results
2. **Flask Backend**
   - REST API endpoint at `/predict`
   - Machine learning model inference
   - Database query for disease information
3. **Machine Learning Model**
   - Algorithm: Random Forest Classifier
   - Input: Binary symptom vector (133 features)

- Output: Disease prediction with ~95% accuracy

Data Flow

```
User Selects Symptoms → React Native App → HTTP POST Request →
Flask API → ML Model → Database Lookup → JSON Response →
Mobile Display
```

Technology Stack

Backend:

- Python 3.8+
- Flask (Web framework)
- scikit-learn (Machine learning)
- pandas & numpy (Data processing)
- joblib (Model serialization)

Frontend:

- React Native 0.76.7
- Expo 52.0.37
- TypeScript
- Axios (HTTP client)
- React Native Modal

Prerequisites

Before starting, ensure you have:

Software Requirements

- **Python 3.8 or higher** - Download
- **Node.js 16 or higher** - Download
- **npm** (comes with Node.js)
- **Text Editor** (VS Code recommended)

Hardware Requirements

- **Computer** running Windows, Mac, or Linux
- **Smartphone** (Android or iOS)
- **WiFi Network** (both devices must be on the same network)

Mobile App

- **Expo Go** installed on your smartphone:
  - Android: Google Play Store
  - iOS: App Store

Backend Setup

Step 1: Organize Backend Files

Create the following folder structure:

```
VIGOR_TRACK/
└── backend/
    ├── app.py
    ├── train.py
    ├── training_og.csv
    ├── description.csv
    ├── precaution.csv
    └── severity.csv
```

Place all backend files (Python scripts and CSV files) in the `backend/` folder.

Step 2: Install Python Dependencies

Open a terminal/command prompt in the `backend/` folder and run:

```
pip install flask flask-cors pandas numpy scikit-learn joblib
```

Package descriptions:

- `flask` - Web server framework
- `flask-cors` - Enable cross-origin requests from mobile
- `pandas` - Data manipulation
- `numpy` - Numerical operations
- `scikit-learn` - Machine learning algorithms
- `joblib` - Model serialization

Troubleshooting: If you get permission errors, try:

```
pip install --user flask flask-cors pandas numpy scikit-learn joblib
```

Step 3: Train the Machine Learning Model

Still in the `backend/` folder, run:

```
python train.py
```

Expected output:

```
Loading training data...
Training Random Forest model...
Model Accuracy: 0.95
Model saved as 'random_forest_model.pkl'
Label encoder saved as 'label_encoder.pkl'
Symptom index saved as 'symptom_index.pkl'
```

What this does:

- Loads training data from CSV files

- Trains a Random Forest classifier
- Creates 3 pickle files (.pkl) needed by the API

Verify: Check that the following files were created:

- `random_forest_model.pkl`
- `label_encoder.pkl`
- `symptom_index.pkl`

Step 4: Find Your Computer's IP Address

You need your computer's local IP address for the mobile app to connect.

Windows:

```
ipconfig
```

Look for "IPv4 Address" under your WiFi adapter. Example: `192.168.1.10`

Mac/Linux:

```
ifconfig
```

Look for "inet" under `en0` or `wlan0`. Example: `192.168.1.10`

Important: Write down this IP address - you'll need it later!

Step 5: Configure Flask for External Connections

Open `app.py` and make the following changes:

1. **Add CORS support** at the top:

```
from flask import Flask, request, jsonify
from flask_cors import CORS  # Add this import
import joblib
import numpy as np
import pandas as pd

app = Flask(__name__)
CORS(app)  # Add this line to enable CORS
```

2. **Update the host** at the bottom:

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
    # Changed host from '127.0.0.1' to '0.0.0.0'
```

Why: By default, Flask only accepts connections from localhost. Setting `host='0.0.0.0'` allows connections from other devices on the network (like your phone).

Step 6: Start the Flask Server

Run the Flask application:

```
python app.py
```

Expected output:

```
* Running on http://0.0.0.0:5000
* Running on http://192.168.1.10:5000 (your actual IP)
* Restarting with stat
* Debugger is active!
```

Important: Keep this terminal window open! The server must run continuously while using the app.

Frontend Setup

Step 7: Organize Frontend Files

Create the following folder structure:

```
VIGOR_TRACK/
└── frontend/
    ├── src/
    │   └── App.tsx
    ├── components/
    │   ├── ThemedText.tsx
    │   ├── ThemedView.tsx
    │   ├── ParallaxScrollView.tsx
    │   ├── Collapsible.tsx
    │   ├── HelloWave.tsx
    │   └── ExternalLink.tsx
    ├── scripts/
    │   └── reset-project.js
    ├── index.tsx
    ├── package.json
    ├── package-lock.json
    ├── app.json
    ├── tsconfig.json
    └── babel.config.js
```

Step 8: Install Node.js Dependencies

Open a **new** terminal in the `frontend/` folder and run:

```
npm install
```

What this does:

- Downloads all dependencies listed in `package.json`
- Creates `node_modules/` folder
- May take 5-10 minutes depending on internet speed

Note: You'll see many packages being installed - this is normal for React Native projects.

Step 9: Update API Endpoint

CRITICAL STEP: You must update the API URL with your computer's IP address.

1. Open `src/App.tsx`
2. Find line ~92 (in the `handleSubmit` function):

```
const response = await axios.post(
  'http://192.168.80.99:5000/predict',
  requestData
);
```

3. Replace `192.168.80.99` with **your computer's IP address** from Step 4:

```
const response = await axios.post(
  'http://YOUR_IP_HERE:5000/predict',
  requestData
);
```

Example:

```
const response = await axios.post(
  'http://192.168.1.10:5000/predict',
  requestData
);
```

4. Save the file

Why: The hardcoded IP address is from the original developer's setup. Your computer will have a different IP address.

Step 10: Install Expo Go on Mobile Device

Download and install **Expo Go** on your smartphone:

- **Android**: Play Store Link
- **iOS**: App Store Link

Open the app after installation - you'll need it in the next step.

Step 11: Start the Expo Development Server

In the `frontend/` folder terminal, run:

```
npx expo start
```

Expected output:

```
▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄
▌▄▄▄▄ ▐▄▄.▐▄▄▄ ▄▄▄▄▌
▌▌ ▐▌▐▌ ▐▌▐▌▐▌▐▌ ▐▌▐▌
▌▐▄▄▄▌▐▄▄▄▌▐▄▄▄▌▐▄▄▄▌

Metro waiting on exp://192.168.1.10:8081

› Press a │ open Android
› Press i │ open iOS simulator
› Press w │ open web
```

```
› Press r │ reload app
```

Keep this terminal open - it must run continuously.

Step 12: Connect Your Mobile Device

Method 1: QR Code (Recommended)

1. Open **Expo Go** app on your phone
2. Tap **"Scan QR Code"**
3. Point your camera at the QR code in the terminal
4. Wait for the app to load (first load may take 1-2 minutes)

Method 2: Manual Connection

1. Ensure phone and computer are on the **same WiFi network**
2. In Expo Go, tap **"Enter URL manually"**
3. Type: `exp://YOUR_IP:8081` (example: `exp://192.168.1.10:8081`)
4. Tap **Connect**

Running the Application

Startup Sequence

1. **Terminal 1** (Backend):
   ```
   cd backend
   python app.py
   ```
2. **Terminal 2** (Frontend):
   ```
   cd frontend
   npx expo start
   ```
3. **Mobile Device**: Scan QR code in Expo Go

Verification Checklist

Before using the app, verify:

- ✅ Flask server running (Terminal 1 shows "Running on...")
- ✅ Expo server running (Terminal 2 shows QR code)
- ✅ Phone connected to same WiFi as computer
- ✅ App loaded on phone (shows "Welcome to VigorTrack!")
- ✅ No error messages in either terminal

Testing & Usage

Using VIGOR TRACK

Once the app loads on your phone:

1. **Welcome Screen**
   - You'll see "□ Welcome to VigorTrack!"
   - Below: "□ Disease predictor □"
2. **Select Symptoms**
   - Tap the input field
   - A dropdown list of 133 symptoms appears
   - Tap any symptom to select it
   - Selected symptoms appear as "bubbles" above the input
3. **Manage Symptoms**
   - Tap the **"X"** on any bubble to remove it
   - Select 3-5 symptoms for best accuracy
   - The model accepts up to 17 symptoms
4. **Get Prediction**
   - Tap the **"Predict Disease"** button
   - Wait 1-2 seconds for processing
   - A modal popup appears with results
5. **View Results**
   - **Predicted Disease**: Name of predicted condition
   - **Description**: What the disease is
   - **Precautions**: Recommended actions
   - Tap **"Close"** to dismiss and try again

Example Test Cases

Test 1: Fungal Infection

- Symptoms: `itching`, `skin_rash`, `nodal_skin_eruptions`
- Expected: Fungal infection

Test 2: Common Cold

- Symptoms: `continuous_sneezing`, `runny_nose`, `congestion`
- Expected: Common Cold or Allergy

Test 3: Diabetes

- Symptoms: `excessive_hunger`, `increased_appetite`, `polyuria`
- Expected: Diabetes

Understanding Results

- **Disclaimer**: The app shows "This is only a prediction... For confirmation, contact a doctor."
- **Accuracy**: Model trained on historical data with ~95% accuracy
- **Limitations**:
  - Only predicts from 41 known diseases
  - Requires accurate symptom selection
  - Not a substitute for professional medical diagnosis

Troubleshooting

Issue 1: "Network Request Failed" Error

Symptoms: App shows error when tapping "Predict Disease"

Causes:

- Phone cannot reach Flask server
- Incorrect IP address in App.tsx
- Firewall blocking connections
- Backend server not running

Solutions:

1. **Verify Flask is running**
   - Check Terminal 1 for "Running on..." message
   - If not running: `python app.py`
2. **Check IP address**
   - In App.tsx, verify IP matches your computer
   - Get current IP: `ipconfig` (Windows) or `ifconfig` (Mac/Linux)
3. **Verify same WiFi network**
   - Phone and computer must be on same network
   - Check WiFi settings on both devices
4. **Test connectivity**
   - Open phone browser
   - Navigate to: `http://YOUR_IP:5000`
   - Should see Flask response or error
5. **Configure firewall**
   Windows:
   - Search "Windows Defender Firewall"
   - Click "Allow an app through firewall"
   - Find Python → Check all boxes → OK

   Mac:
   - System Preferences → Security & Privacy → Firewall
   - Firewall Options → Add Python → Allow

Issue 2: Python Module Not Found

Symptoms: `ModuleNotFoundError: No module named 'flask'` (or pandas, numpy, etc.)

Solution:

```
pip install flask flask-cors pandas numpy scikit-learn joblib
```

If still failing, use virtual environment:

```
# Create virtual environment<a></a>
python -m venv venv

# Activate it<a></a>
# Windows:<a></a>
venv\Scripts\activate
# Mac/Linux:<a></a>
source venv/bin/activate

# Install packages<a></a>
pip install flask flask-cors pandas numpy scikit-learn joblib
```

Issue 3: Model Files Not Found

Symptoms: Flask error: `FileNotFoundError: random_forest_model.pkl not found`

Solution:

1. Navigate to backend folder
2. Run training script:

   ```
   python train.py
   ```

3. Verify 3 `.pkl` files created
4. Restart Flask server

Issue 4: Unable to Resolve Module (React Native)

Symptoms: Red error screen on phone: "Unable to resolve module..."

Solution:

```
# Delete cache and dependencies<a></a>
rm -rf node_modules
rm package-lock.json

# Reinstall<a></a>
npm install

# Clear Metro bundler cache<a></a>
npx expo start -c
```

Issue 5: Expo App Stuck on Splash Screen

Symptoms: App shows Expo logo but won't load

Solutions:

1. **Reload app**
   - Shake phone
   - Tap "Reload"
2. **Restart Metro**
   - In terminal, press `Ctrl+C`
   - Run: `npx expo start -c` (clears cache)
3. **Check terminal for errors**
   - Look for red error messages
   - Fix any JavaScript errors in code

Issue 6: Port 5000 Already in Use

Symptoms: `OSError: [Errno 48] Address already in use`

Solutions:

Option A: Kill process on port 5000

Windows:

```
netstat -ano | findstr :5000
taskkill /PID &lt;PID&gt; /F
```

Mac/Linux:

```
lsof -ti:5000 | xargs kill -9
```

Option B: Use different port

In `app.py`:

```
app.run(host='0.0.0.0', port=5001, debug=True)
```

In `App.tsx`:

```
'http://YOUR_IP:5001/predict'
```

Issue 7: QR Code Won't Scan

Symptoms: Expo Go camera doesn't recognize QR code

Solutions:

1. **Grant camera permission**
   - Phone Settings → Apps → Expo Go → Permissions
   - Enable Camera

2. **Enter URL manually**
   - In Expo Go: "Enter URL manually"
   - Type: `exp://YOUR_IP:8081`
3. **Increase QR code size**
   - In terminal, press `Ctrl+C`
   - Run: `npx expo start --tunnel`

Issue 8: IP Address Keeps Changing

Symptoms: App works initially, then stops after reconnecting to WiFi

Solutions:

Option A: Set static IP (Recommended)

1. Access router admin panel (usually 192.168.1.1)
2. Find DHCP settings
3. Reserve/assign static IP to your computer's MAC address

Option B: Use ngrok (For remote testing)

```
# Install ngrok<a></a>
npm install -g ngrok

# Expose Flask<a></a>
ngrok http 5000

# Use ngrok URL in App.tsx<a></a>
# Example: http://abc123.ngrok.io/predict<a></a>
```

Issue 9: Slow Performance

Symptoms: App takes long time to load or predict

Solutions:

1. **Check network speed**
   - Ensure strong WiFi signal
   - Move closer to router
2. **Optimize model**
   - Reduce model complexity in `train.py`
   - Use fewer features
3. **Enable production mode**

   ```
   export FLASK_ENV=production
   python app.py
   ```

Issue 10: Predictions Are Incorrect

Symptoms: Model returns wrong or unexpected diseases

Causes:

- Symptom name mismatch
- Insufficient symptoms selected
- Model needs retraining

Solutions:

1. **Verify symptom format**
   - Must use underscores: `skin_rash` not "skin rash"
   - Must match exactly: `continuous_sneezing` not "sneezing"
2. **Select more symptoms**
   - Use at least 3-5 symptoms
   - Be as specific as possible
3. **Retrain model**

   ```
   cd backend
   python train.py
   ```

Development Tips

Hot Reload

Frontend: Changes to `App.tsx` automatically reload on phone

- No need to restart Expo server
- If auto-reload fails: Shake phone → Reload

Backend: Flask auto-reloads in debug mode

- Changes to `app.py` restart server automatically
- Watch terminal for "Restarting with stat" message

Logging & Debugging

Backend Logs:

- Terminal 1 shows all API requests
- Example: `POST /predict HTTP/1.1" 200`
- Add print statements in `app.py` for debugging

Frontend Logs:

- Terminal 2 shows React Native errors
- Phone console: Shake → "Open Console"
- Add `console.log()` in `App.tsx` for debugging

Testing on Multiple Devices

- Multiple phones can connect to same Expo server
- Each phone scans same QR code
- Useful for cross-device testing
- All devices must be on same WiFi

Code Organization Tips

Backend:

- Keep CSV files in `backend/data/` folder
- Store models in `backend/models/` folder
- Create separate file for model training
- Use environment variables for configuration

Frontend:

- Create separate components for screens
- Use React Context for state management
- Store API calls in `services/api.ts`
- Create constants file for configuration

Performance Optimization

Backend:

- Use Redis caching for frequent predictions
- Implement connection pooling
- Enable gzip compression
- Deploy to cloud for better performance

Frontend:

- Lazy load components
- Optimize images
- Implement pagination for symptom list
- Cache API responses

Production Deployment

Backend:

- Deploy to: Heroku, AWS, Railway, or PythonAnywhere
- Use production WSGI server (Gunicorn)
- Set up environment variables
- Enable HTTPS

Frontend:

- Build standalone APK (Android):

  ```
  eas build --platform android
  ```

- Build IPA (iOS):

```
eas build --platform ios
```

- Submit to app stores
- Requires Expo EAS account

Security Best Practices

1. **Never commit sensitive data**
   - Add `.env` to `.gitignore`
   - Use environment variables for secrets
2. **Validate user input**
   - Check symptom names on backend
   - Limit request size
   - Implement rate limiting
3. **Secure API**
   - Add authentication (JWT tokens)
   - Use HTTPS in production
   - Implement API rate limiting
4. **Data privacy**
   - Don't store user health data
   - Add privacy policy
   - Comply with HIPAA if applicable

Next Steps

Feature Enhancements

1. **User Authentication**
   - Firebase Auth or custom JWT
   - User profiles
   - Prediction history
2. **Advanced Features**
   - Symptom search/filter
   - Voice input for symptoms
   - Multi-language support
   - Confidence scores
   - Similar disease suggestions
3. **Data Improvements**
   - More diseases (expand beyond 41)
   - More symptoms
   - Regular model retraining
   - Add severity levels

4. **UI/UX Enhancements**
    - Dark mode
    - Custom themes
    - Animations
    - Onboarding tutorial
    - Accessibility features
5. **Integration**
    - Doctor consultation booking
    - Pharmacy integration
    - Health tracking
    - Medical records

Learning Resources

React Native:

- Official Docs
- Expo Documentation

Flask:

- Flask Docs
- Flask Mega-Tutorial

Machine Learning:

- scikit-learn Docs
- Kaggle Tutorials

Appendix

Project File Structure

```
VIGOR_TRACK/
├── backend/
│   ├── app.py                      # Flask API server
│   ├── train.py                    # ML training script
│   ├── training_og.csv             # Training data (4920 records)
│   ├── description.csv             # Disease descriptions (41)
│   ├── precaution.csv              # Disease precautions
│   ├── severity.csv                # Symptom weights (133)
│   ├── random_forest_model.pkl     # Trained model (generated)
│   ├── label_encoder.pkl           # Label encoder (generated)
│   └── symptom_index.pkl           # Symptom index (generated)
│
└── frontend/
    ├── src/
    │   └── App.tsx                 # Main app component
    ├── components/
    │   ├── ThemedText.tsx          # Text component
    │   ├── ThemedView.tsx          # View component
    │   ├── ParallaxScrollView.tsx  # Scroll component
    │   ├── Collapsible.tsx         # Collapsible component
    │   ├── HelloWave.tsx           # Wave animation
    │   └── ExternalLink.tsx        # Link component
```

```
    ├── scripts/
    │   └── reset-project.js        # Reset utility
    ├── node_modules/               # Dependencies (generated)
    ├── index.tsx                   # Entry point
    ├── package.json                # Dependencies
    ├── package-lock.json           # Locked versions
    ├── app.json                    # Expo config
    ├── tsconfig.json               # TypeScript config
    └── babel.config.js             # Babel config
```

Command Reference

Backend Commands:

```
pip install flask flask-cors pandas numpy scikit-learn joblib
python train.py
python app.py
```

Frontend Commands:

```
npm install
npx expo start
npx expo start -c              # Clear cache
npx expo start --tunnel        # Use tunnel connection
```

Network Commands:

```
# Get IP address<a></a>
ipconfig          # Windows
ifconfig          # Mac/Linux

# Check port usage<a></a>
netstat -ano | findstr :5000   # Windows
lsof -ti:5000                  # Mac/Linux

# Test connectivity<a></a>
ping 192.168.1.10
curl http://192.168.1.10:5000
```

Environment Variables

Create `.env` file in backend/:

```
FLASK_APP=app.py
FLASK_ENV=development
HOST=0.0.0.0
PORT=5000
```

Load in `app.py`:

```
from dotenv import load_dotenv
import os

load_dotenv()
app.run(host=os.getenv('HOST'), port=int(os.getenv('PORT')))
```

Document Version: 1.0
Last Updated: October 2025
Project: VIGOR TRACK
License: MIT (or your license)

For questions or issues, please contact the development team or open an issue on the project repository.